

```
from google.colab import files
uploaded = files.upload()
```

Pilih File kfood_dataset.zip
kfood_dataset.zip(application/x-zip-compressed) - 207825962 bytes, last modified: 16/11/2025 - 100% done
 Saving kfood_dataset.zip to kfood_dataset.zip

Aktifkan notifikasi browser di
 Setelan untuk mendapatkan
 pemberitahuan saat eksekusi
 selesai

Oke Lain kali

```
import zipfile
import os

zip_path = 'kfood_dataset.zip' # ganti sesuai nama file kamu

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content/korean_food_dataset')

print("Selesai diekstrak!")
```

Selesai diekstrak!

```
!ls /content/korean_food_dataset
```

content

```
!ls /content/korean_food_dataset/content
```

korean_food_dataset

```
dataset_path = "/content/korean_food_dataset/content/korean_food_dataset"
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
```

```
train_generator = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(300, 300),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_generator = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(300, 300),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)
```

Found 4000 images belonging to 25 classes.
 Found 1000 images belonging to 25 classes.

```
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

base_model = EfficientNetB3(
```

```

        include_top=False,
        weights='imagenet',
        input_shape=(300,300,3)
    )

    base_model.trainable = False

```

Aktifkan notifikasi browser di
Setelan untuk mendapatkan
pemberitahuan saat eksekusi
selesai

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb3_notop.h5
43941136/43941136 — 0s 0us/step

```

x = GlobalAveragePooling2D()(base_model.output)
x = Dropout(0.4)(x)
output = Dense(train_generator.num_classes, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)

```

```

model.compile(
    optimizer=Adam(0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

```

```

history1 = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=15
)

```

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
self._warn_if_super_not_called()

```

Epoch 1/15
125/125 ————— 183s 1s/step - accuracy: 0.0395 - loss: 3.2929 - val_accuracy: 0.0400 - val_loss: 3.2186
Epoch 2/15
125/125 ————— 109s 871ms/step - accuracy: 0.0366 - loss: 3.2441 - val_accuracy: 0.0560 - val_loss: 3.2349
Epoch 3/15
125/125 ————— 110s 880ms/step - accuracy: 0.0534 - loss: 3.2421 - val_accuracy: 0.0620 - val_loss: 3.1979
Epoch 4/15
125/125 ————— 110s 880ms/step - accuracy: 0.0468 - loss: 3.2179 - val_accuracy: 0.0510 - val_loss: 3.2225
Epoch 5/15
125/125 ————— 110s 885ms/step - accuracy: 0.0552 - loss: 3.2234 - val_accuracy: 0.0490 - val_loss: 3.2142
Epoch 6/15
125/125 ————— 109s 870ms/step - accuracy: 0.0526 - loss: 3.2258 - val_accuracy: 0.0550 - val_loss: 3.1955
Epoch 7/15
125/125 ————— 109s 876ms/step - accuracy: 0.0608 - loss: 3.2229 - val_accuracy: 0.0470 - val_loss: 3.1968
Epoch 8/15
125/125 ————— 109s 870ms/step - accuracy: 0.0510 - loss: 3.2191 - val_accuracy: 0.0560 - val_loss: 3.1879
Epoch 9/15
125/125 ————— 109s 877ms/step - accuracy: 0.0476 - loss: 3.2240 - val_accuracy: 0.0520 - val_loss: 3.1984
Epoch 10/15
125/125 ————— 109s 875ms/step - accuracy: 0.0522 - loss: 3.2365 - val_accuracy: 0.0590 - val_loss: 3.1860
Epoch 11/15
125/125 ————— 115s 919ms/step - accuracy: 0.0546 - loss: 3.2227 - val_accuracy: 0.0630 - val_loss: 3.2053
Epoch 12/15
125/125 ————— 110s 878ms/step - accuracy: 0.0586 - loss: 3.2294 - val_accuracy: 0.0570 - val_loss: 3.2044
Epoch 13/15
125/125 ————— 109s 872ms/step - accuracy: 0.0541 - loss: 3.2418 - val_accuracy: 0.0580 - val_loss: 3.1984
Epoch 14/15
125/125 ————— 110s 881ms/step - accuracy: 0.0570 - loss: 3.2249 - val_accuracy: 0.0650 - val_loss: 3.1983
Epoch 15/15
125/125 ————— 107s 860ms/step - accuracy: 0.0587 - loss: 3.2227 - val_accuracy: 0.0600 - val_loss: 3.1873

```

```

base_model.trainable = True

model.compile(
    optimizer=Adam(1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history2 = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=20
)

```

)

Epoch 1/20
 125/125 ————— 271s 1s/step - accuracy: 0.0455 - loss: 3.3934 - val_accuracy: 0.0410 - v
 Epoch 2/20
 125/125 ————— 124s 995ms/step - accuracy: 0.1117 - loss: 3.0883 - val_accuracy: 0.0410
 Epoch 3/20
 125/125 ————— 124s 992ms/step - accuracy: 0.1491 - loss: 2.9333 - val_accuracy: 0.0490
 Epoch 4/20
 125/125 ————— 124s 995ms/step - accuracy: 0.2120 - loss: 2.7377 - val_accuracy: 0.1380
 Epoch 5/20
 125/125 ————— 124s 995ms/step - accuracy: 0.2638 - loss: 2.5992 - val_accuracy: 0.2410 - val_loss: 2.7087
 Epoch 6/20
 125/125 ————— 125s 999ms/step - accuracy: 0.3399 - loss: 2.3981 - val_accuracy: 0.3530 - val_loss: 2.4047
 Epoch 7/20
 125/125 ————— 126s 1s/step - accuracy: 0.4155 - loss: 2.1862 - val_accuracy: 0.3650 - val_loss: 2.3437
 Epoch 8/20
 125/125 ————— 126s 1s/step - accuracy: 0.4620 - loss: 2.0640 - val_accuracy: 0.4150 - val_loss: 2.2112
 Epoch 9/20
 125/125 ————— 125s 1s/step - accuracy: 0.5058 - loss: 1.9190 - val_accuracy: 0.4510 - val_loss: 2.0306
 Epoch 10/20
 125/125 ————— 145s 1s/step - accuracy: 0.5285 - loss: 1.7779 - val_accuracy: 0.4680 - val_loss: 2.0105
 Epoch 11/20
 125/125 ————— 124s 993ms/step - accuracy: 0.5804 - loss: 1.6372 - val_accuracy: 0.5610 - val_loss: 1.6837
 Epoch 12/20
 125/125 ————— 125s 998ms/step - accuracy: 0.6240 - loss: 1.4811 - val_accuracy: 0.6220 - val_loss: 1.4974
 Epoch 13/20
 125/125 ————— 125s 1000ms/step - accuracy: 0.6407 - loss: 1.3809 - val_accuracy: 0.6490 - val_loss: 1.4167
 Epoch 14/20
 125/125 ————— 125s 1s/step - accuracy: 0.6682 - loss: 1.2946 - val_accuracy: 0.5600 - val_loss: 1.6846
 Epoch 15/20
 125/125 ————— 124s 988ms/step - accuracy: 0.6768 - loss: 1.1970 - val_accuracy: 0.6060 - val_loss: 1.4754
 Epoch 16/20
 125/125 ————— 124s 988ms/step - accuracy: 0.6986 - loss: 1.1310 - val_accuracy: 0.6450 - val_loss: 1.3108
 Epoch 17/20
 125/125 ————— 123s 986ms/step - accuracy: 0.7131 - loss: 1.0657 - val_accuracy: 0.6120 - val_loss: 1.3925
 Epoch 18/20
 125/125 ————— 125s 995ms/step - accuracy: 0.7376 - loss: 1.0002 - val_accuracy: 0.6890 - val_loss: 1.1856
 Epoch 19/20
 125/125 ————— 124s 989ms/step - accuracy: 0.7569 - loss: 0.9284 - val_accuracy: 0.6460 - val_loss: 1.3387
 Epoch 20/20
 125/125 ————— 124s 993ms/step - accuracy: 0.7638 - loss: 0.8954 - val_accuracy: 0.6930 - val_loss: 1.1334

Aktifkan notifikasi browser di
 Setelan untuk mendapatkan
 pemberitahuan saat eksekusi
 selesai

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,

    rotation_range=40,      # lebih besar
    zoom_range=0.3,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    horizontal_flip=True,
    brightness_range=[0.7, 1.3], # tambahan
)
```

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

callbacks = [
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5)
]
```

```
target_size=(380, 380)
input_shape=(380, 380, 3)
```

```
base_model = EfficientNetB3(
    include_top=False,
    weights='imagenet',
    input_shape=(380,380,3)
)
```

```
x = GlobalAveragePooling2D()(base_model.output)
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(train_generator.num_classes, activation='softmax')(x)
```

Aktifkan notifikasi browser di
Setelan untuk mendapatkan
pemberitahuan saat eksekusi
selesai

```
history2 = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=20,
    callbacks=callbacks
)
```

```
Epoch 1/20
125/125 ————— 126s 1s/step - accuracy: 0.7752 - loss: 0.8551 - val_accuracy: 0.7270 - val_loss: 1.0176 - learning
Epoch 2/20
125/125 ————— 124s 992ms/step - accuracy: 0.7810 - loss: 0.8199 - val_accuracy: 0.7310 - val_loss: 0.9855 - learn
Epoch 3/20
125/125 ————— 124s 990ms/step - accuracy: 0.7986 - loss: 0.7647 - val_accuracy: 0.7250 - val_loss: 1.0083 - learn
Epoch 4/20
125/125 ————— 124s 988ms/step - accuracy: 0.8030 - loss: 0.7295 - val_accuracy: 0.6830 - val_loss: 1.1755 - learn
Epoch 5/20
125/125 ————— 123s 987ms/step - accuracy: 0.8041 - loss: 0.6949 - val_accuracy: 0.6880 - val_loss: 1.1171 - learn
Epoch 6/20
125/125 ————— 123s 986ms/step - accuracy: 0.8353 - loss: 0.6567 - val_accuracy: 0.7510 - val_loss: 0.9075 - learn
Epoch 7/20
125/125 ————— 125s 998ms/step - accuracy: 0.8284 - loss: 0.6375 - val_accuracy: 0.7160 - val_loss: 1.0651 - learn
Epoch 8/20
125/125 ————— 124s 991ms/step - accuracy: 0.8328 - loss: 0.6243 - val_accuracy: 0.7060 - val_loss: 1.0862 - learn
Epoch 9/20
125/125 ————— 124s 989ms/step - accuracy: 0.8509 - loss: 0.5886 - val_accuracy: 0.6530 - val_loss: 1.2587 - learn
Epoch 10/20
125/125 ————— 123s 987ms/step - accuracy: 0.8552 - loss: 0.5243 - val_accuracy: 0.7120 - val_loss: 1.0526 - learn
Epoch 11/20
125/125 ————— 124s 989ms/step - accuracy: 0.8629 - loss: 0.5230 - val_accuracy: 0.7720 - val_loss: 0.8281 - learn
Epoch 12/20
125/125 ————— 124s 989ms/step - accuracy: 0.8757 - loss: 0.4769 - val_accuracy: 0.7670 - val_loss: 0.8141 - learn
Epoch 13/20
125/125 ————— 124s 991ms/step - accuracy: 0.8723 - loss: 0.4773 - val_accuracy: 0.7710 - val_loss: 0.8076 - learn
Epoch 14/20
125/125 ————— 124s 990ms/step - accuracy: 0.8743 - loss: 0.4652 - val_accuracy: 0.7490 - val_loss: 0.9306 - learn
Epoch 15/20
125/125 ————— 124s 991ms/step - accuracy: 0.8935 - loss: 0.4192 - val_accuracy: 0.7590 - val_loss: 0.8453 - learn
Epoch 16/20
125/125 ————— 124s 989ms/step - accuracy: 0.8976 - loss: 0.3885 - val_accuracy: 0.7400 - val_loss: 0.9478 - learn
Epoch 17/20
125/125 ————— 124s 989ms/step - accuracy: 0.8924 - loss: 0.3999 - val_accuracy: 0.7770 - val_loss: 0.8165 - learn
Epoch 18/20
125/125 ————— 124s 988ms/step - accuracy: 0.9042 - loss: 0.3889 - val_accuracy: 0.7640 - val_loss: 0.8401 - learn
Epoch 19/20
125/125 ————— 124s 993ms/step - accuracy: 0.9077 - loss: 0.3662 - val_accuracy: 0.7900 - val_loss: 0.7682 - learn
Epoch 20/20
125/125 ————— 124s 993ms/step - accuracy: 0.9082 - loss: 0.3718 - val_accuracy: 0.7810 - val_loss: 0.7812 - learn
```

```
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
```

```
loss = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)

model.compile(
    optimizer=Adam(1e-5),
    loss=loss,
    metrics=['accuracy']
)
```

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
```

)

```

for layer in base_model.layers[:200]:
    layer.trainable = False
for layer in base_model.layers[200:]:
    layer.trainable = True

```

Aktifkan notifikasi browser di
Setelan untuk mendapatkan
pemberitahuan saat eksekusi
selesai

```

callbacks = [
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=3,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=7,
        restore_best_weights=True
    )
]

```

```

history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=20,
    callbacks=callbacks
)

```

```

Epoch 1/20
125/125 ————— 250s 1s/step - accuracy: 0.9039 - loss: 1.0731 - val_accuracy: 0.7240 - val_loss: 1.5032 - learning
Epoch 2/20
125/125 ————— 123s 986ms/step - accuracy: 0.9286 - loss: 0.9880 - val_accuracy: 0.7460 - val_loss: 1.4198 - learn
Epoch 3/20
125/125 ————— 124s 990ms/step - accuracy: 0.9290 - loss: 0.9695 - val_accuracy: 0.7760 - val_loss: 1.3262 - learn
Epoch 4/20
125/125 ————— 123s 987ms/step - accuracy: 0.9275 - loss: 0.9546 - val_accuracy: 0.7860 - val_loss: 1.3172 - learn
Epoch 5/20
125/125 ————— 125s 996ms/step - accuracy: 0.9405 - loss: 0.9301 - val_accuracy: 0.7540 - val_loss: 1.3967 - learn
Epoch 6/20
125/125 ————— 123s 985ms/step - accuracy: 0.9393 - loss: 0.9248 - val_accuracy: 0.7690 - val_loss: 1.3605 - learn
Epoch 7/20
125/125 ————— 0s 815ms/step - accuracy: 0.9442 - loss: 0.9078
Epoch 7: ReduceLROnPlateau reducing learning rate to 1.9999999494757505e-06.
125/125 ————— 124s 986ms/step - accuracy: 0.9442 - loss: 0.9078 - val_accuracy: 0.7720 - val_loss: 1.3600 - learn
Epoch 8/20
125/125 ————— 123s 987ms/step - accuracy: 0.9461 - loss: 0.9084 - val_accuracy: 0.7900 - val_loss: 1.2771 - learn
Epoch 9/20
125/125 ————— 123s 984ms/step - accuracy: 0.9502 - loss: 0.8952 - val_accuracy: 0.7850 - val_loss: 1.2784 - learn
Epoch 10/20
125/125 ————— 124s 992ms/step - accuracy: 0.9593 - loss: 0.8849 - val_accuracy: 0.7840 - val_loss: 1.2762 - learn
Epoch 11/20
125/125 ————— 124s 992ms/step - accuracy: 0.9527 - loss: 0.8916 - val_accuracy: 0.7850 - val_loss: 1.2823 - learn
Epoch 12/20
125/125 ————— 124s 990ms/step - accuracy: 0.9502 - loss: 0.9076 - val_accuracy: 0.7850 - val_loss: 1.2879 - learn
Epoch 13/20
125/125 ————— 124s 993ms/step - accuracy: 0.9522 - loss: 0.8863 - val_accuracy: 0.7940 - val_loss: 1.2670 - learn
Epoch 14/20
125/125 ————— 124s 989ms/step - accuracy: 0.9496 - loss: 0.8845 - val_accuracy: 0.7870 - val_loss: 1.2706 - learn
Epoch 15/20
125/125 ————— 124s 991ms/step - accuracy: 0.9480 - loss: 0.8838 - val_accuracy: 0.7850 - val_loss: 1.2819 - learn
Epoch 16/20
125/125 ————— 124s 991ms/step - accuracy: 0.9486 - loss: 0.8897 - val_accuracy: 0.8000 - val_loss: 1.2641 - learn
Epoch 17/20
125/125 ————— 124s 989ms/step - accuracy: 0.9485 - loss: 0.8926 - val_accuracy: 0.7900 - val_loss: 1.2756 - learn
Epoch 18/20
125/125 ————— 123s 986ms/step - accuracy: 0.9534 - loss: 0.8858 - val_accuracy: 0.7920 - val_loss: 1.2668 - learn
Epoch 19/20
125/125 ————— 0s 821ms/step - accuracy: 0.9613 - loss: 0.8663
Epoch 19: ReduceLROnPlateau reducing learning rate to 3.99999989900971e-07.
125/125 ————— 123s 985ms/step - accuracy: 0.9612 - loss: 0.8664 - val_accuracy: 0.7930 - val_loss: 1.2642 - learn
Epoch 20/20
125/125 ————— 124s 990ms/step - accuracy: 0.9494 - loss: 0.8920 - val_accuracy: 0.7810 - val_loss: 1.2747 - learn

```

```
model.save("efficientnet_korean_food.keras")
print("Model saved!")
```

Model saved!

Aktifkan notifikasi browser di
Setelan untuk mendapatkan
pemberitahuan saat eksekusi
selesai

```
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
```

```
loss = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)

model.compile(
    optimizer=Adam(1e-5),
    loss=loss,
    metrics=['accuracy']
)
```

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=20,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
```

```
for layer in base_model.layers[:200]:
    layer.trainable = False
for layer in base_model.layers[200:]:
    layer.trainable = True
```

```
callbacks = [
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=3,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=7,
        restore_best_weights=True
    )
]
```

```
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=20,
    callbacks=callbacks
)
```

```
Epoch 1/20
125/125 ————— 251s 1s/step - accuracy: 0.9502 - loss: 0.8829 - val_accuracy: 0.7070 - val_loss: 1.4715 - learning
Epoch 2/20
125/125 ————— 124s 988ms/step - accuracy: 0.9633 - loss: 0.8680 - val_accuracy: 0.7690 - val_loss: 1.3298 - learn
Epoch 3/20
125/125 ————— 123s 987ms/step - accuracy: 0.9639 - loss: 0.8660 - val_accuracy: 0.7880 - val_loss: 1.2814 - learn
Epoch 4/20
9/125 ————— 1:38 850ms/step - accuracy: 0.9594 - loss: 0.8599
```



Aktifkan notifikasi browser di
Setelan untuk mendapatkan
pemberitahuan saat eksekusi
selesai